

Linear Regression

김성영교수
국립금오공과대학교
컴퓨터공학부

학습 내용

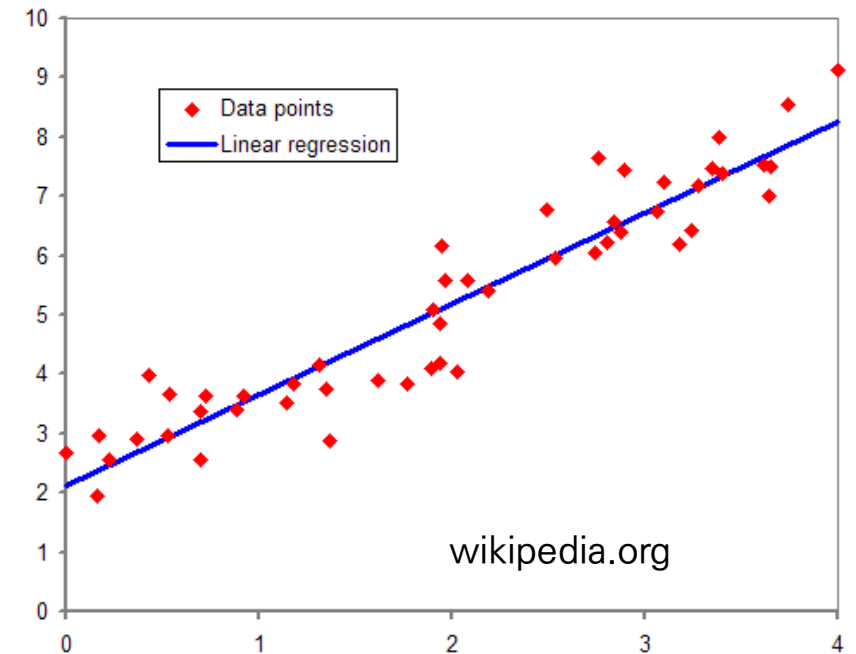
- 회귀 분석이란 무엇인지 설명할 수 있다.
- 선형 회귀 분석의 목적에 대해 설명할 수 있다.
- 손실 함수와 비용 함수에 대해 설명할 수 있다.
- 최소 제곱법을 사용하여 선형 회귀 모델을 추정할 수 있다.

회귀regression

- 자료 분석을 위한 통계학적 방법

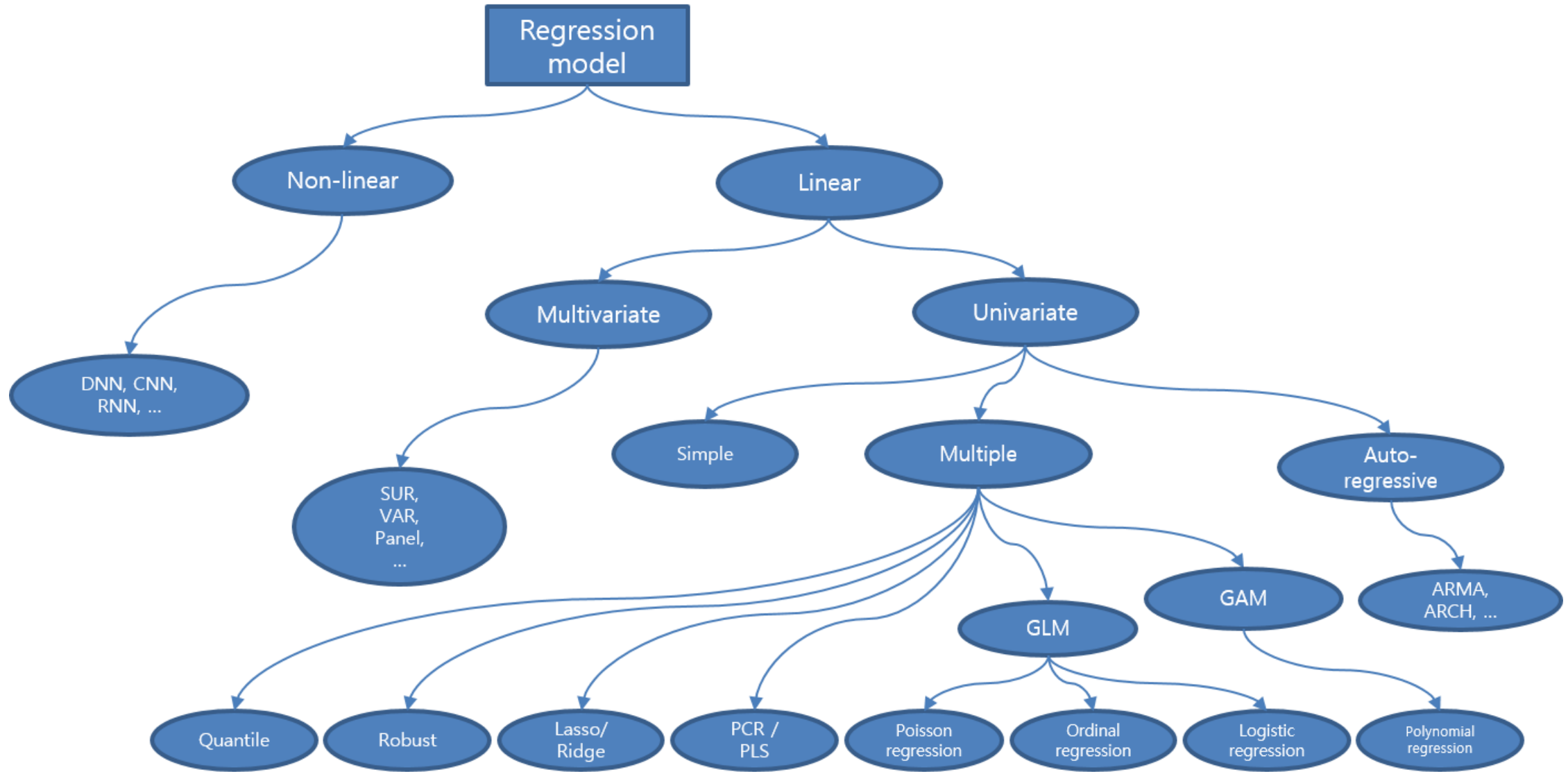
- 주어진 자료의 경향성 분석

- 입력 변수 X 에 대응하는 반응 변수 Y 의 조건부 평균 또는 함수적 관계 추정



회귀regression

- 프랜시스 골턴(Francis Galton)의 '평균으로의 회귀(Regression toward the mean)' 현상에서 유래
 - 극단적인 관측치가 결국 중심(평균) 경향성으로 되돌아간다는 현상적 속성
- 머신러닝 태스크에서의 사용 맥락
 - 분류(Classification)와 대비되는 '연속형 실수값을 예측하는 작업 유형(Task)'
- 종류
 - 고전적 선형(linear) 회귀, 커널(kernel) 회귀, 다항(polynomial) 회귀, GAM(Generalize Additive Model), 로지스틱(logistic) 회귀 등



https://danbi-ncsoft.github.io/study/2018/05/04/study-regression_model_summary.html

회귀 모델

기울기 y절편 찾는 것 이 목표

- 회귀 관계를 구체화한 수학적 함수식

- 수학적 모델을 가정하고 주어진 자료들을 사용하여 모델의 파라미터 결정

- 일반식: $Y = f(X; \beta) + \epsilon$

- X : 독립변수, Y : 종속변수

- ϵ : 측정상의 오차나 현실적인 한계로 인해 발생하는 불확실성 (평균이 0이고 분산이 일정한 정규 분포)

아닐 수 도 있음

모델의 추정 = (모델의 관계식 추정) + (파라미터 추정)

선형 회귀 linear regression

- 한 개 이상의 독립 변수(x)와 종속 변수(y) 사이의 선형 상관 관계를 정량적으로 추정하는 회귀 방법
 - 독립 변수: 'x값이 변함에 따라 y값도 변한다' 정의 하에서 독립적으로 변할 수 있는 x값
 - 종속 변수: 독립 변수에 따라 종속적으로 변하는 값
- 단순 선형 회귀와 다중 선형 회귀
 - 단순 선형 회귀 single linear regression : 1개의 독립 변수를 사용하는 선형 회귀
 - 다중 선형 회귀 multiple linear regression : 2개 이상의 독립 변수를 사용하는 선형 회귀

$$H(x) = \hat{y} = wx + b$$

w와 b를 찾는 것 이 목표

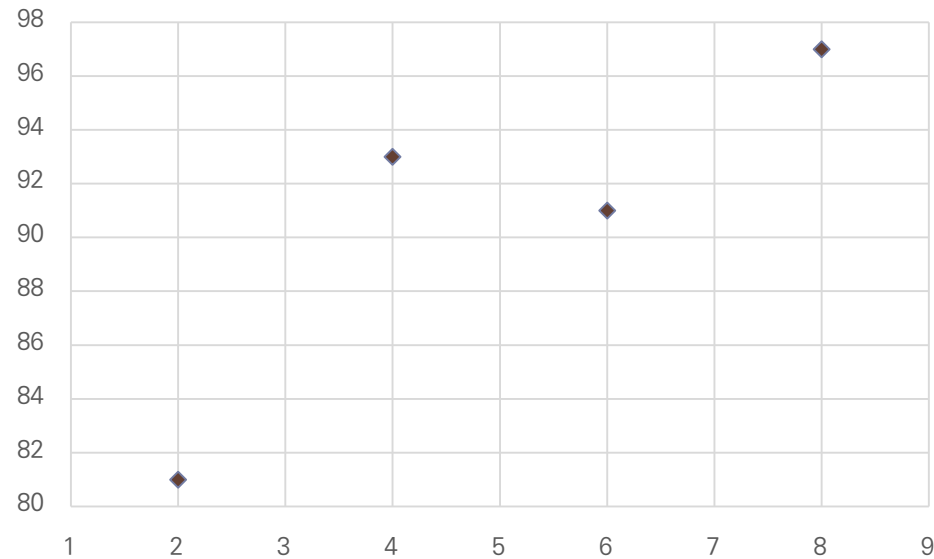
w: 가중치(weight)

b: 바이어스(bias)

예제: 공부시간에 따른 성적 추정

- 학습 데이터 training data

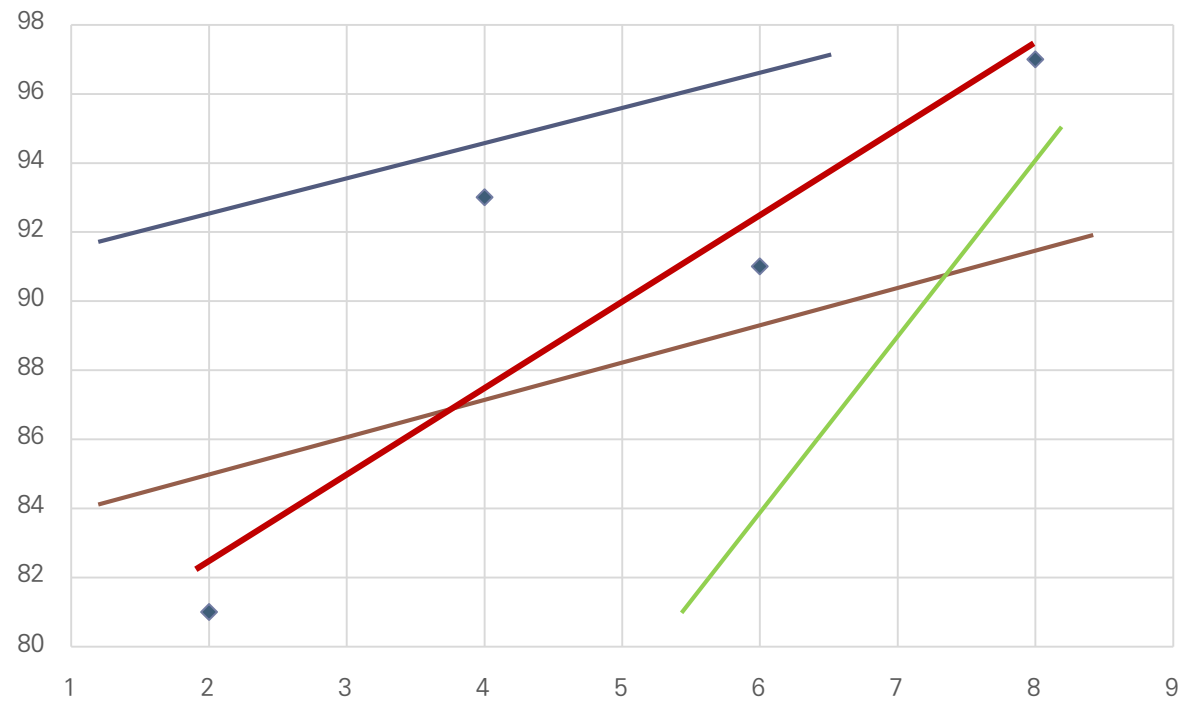
X	공부한 시간	2시간	4시간	6시간	8시간
Y	성적	81점	93점	91점	97점



모두의 딥러닝, 조태호 저, 길벗

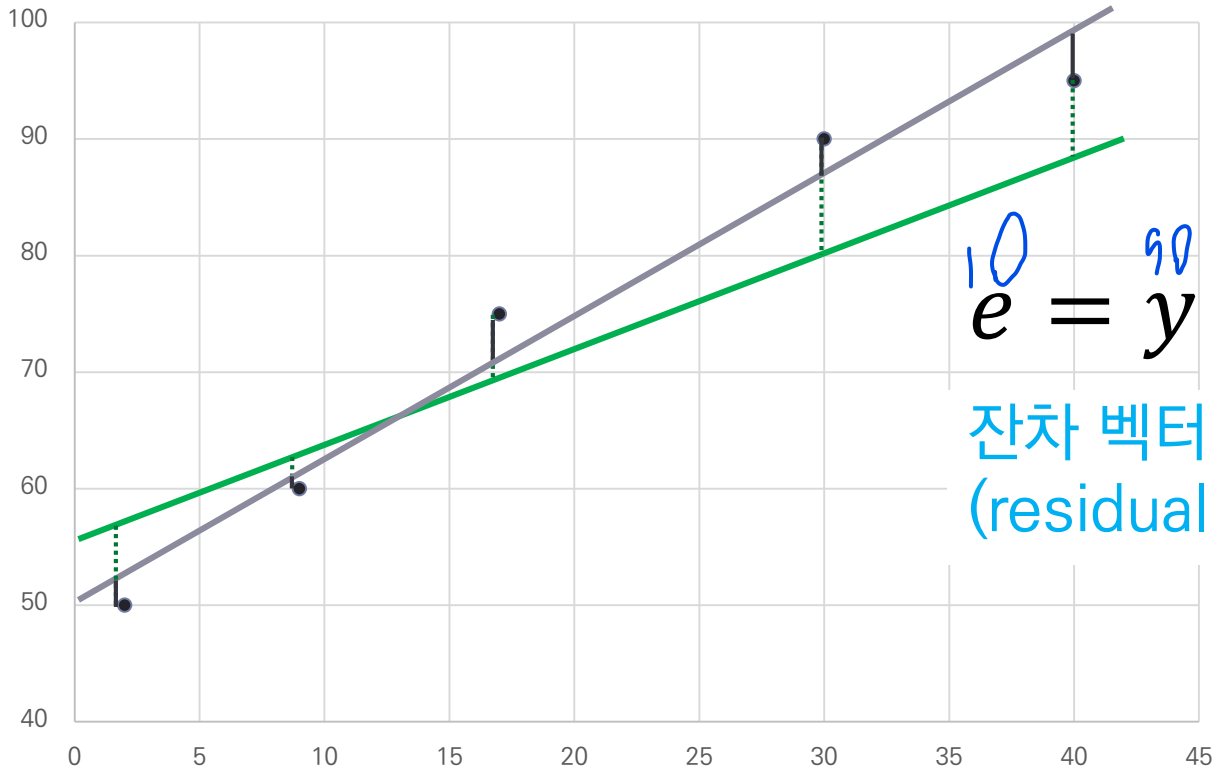
가설hypothesis

$$H(x) = wx + b$$



hypothesis (linear model)를 세우고
가정
훈련 집합에 가장 적합한 것을 찾는 과정

참고: 잔차 벡터 residual vector



$$H(x) = \hat{y} = wx + b$$

$$e = y - \hat{y} = y - (wx + b)$$

y : 실제 점수
 $\hat{y}$: 예측된 점수
 e : 잔차 (오차)

잔차 제곱합(RSS: residual sum of squares)

$$RSS = \sum e^T e$$

잔차가 여러 개 있을 때, 그걸 모두 제곱해서 더한 값

Ordinary Least Squares (OLS) (1)

최소 제곱법

- 결정론적 회귀 분석 방법

- 단순히 독립 변수 x 에 대해 대응하는 종속 변수 y 를 계산하는 함수를 만드는 과정

- 미분을 통해 잔차제곱합(RSS: Residual Sum of Squares)를 최소화하는 파라미터 결정

- 잔차 벡터 (residual vector) $\mathbf{e} = \mathbf{y} - \hat{\mathbf{y}} = \mathbf{y} - \mathbf{X}\mathbf{w}$

$\mathbf{w}$ 는 $n+1$ 차원 벡터
 $\mathbf{X}$ 는 m 개 샘플을 갖는 $(m \times (n+1))$ 행렬

- 잔차 제곱합 (residual sum of squares)

$$\mathbf{e}^T \mathbf{e} = (\mathbf{y} - \mathbf{X}\mathbf{w})^T (\mathbf{y} - \mathbf{X}\mathbf{w}) = \mathbf{y}^T \mathbf{y} - 2\mathbf{y}^T \mathbf{X}\mathbf{w} + \mathbf{w}^T \mathbf{X}^T \mathbf{X} \mathbf{w}$$

- 잔차 제곱합의 기울기 벡터

$$\frac{d\mathbf{e}^T \mathbf{e}}{d\mathbf{w}} = -2\mathbf{X}^T \mathbf{y} + 2\mathbf{X}^T \mathbf{X} \mathbf{w}$$

Ordinary Least Squares (OLS) (2)

- 최적화 조건 흐름만 이해

$$\frac{d\mathbf{e}^T \mathbf{e}}{d\mathbf{w}} = 0$$

$$\frac{d\mathbf{e}^T \mathbf{e}}{d\mathbf{w}} = -2\mathbf{yX}^T + 2\mathbf{XX}^T \mathbf{w}$$

$$\mathbf{XX}^T \mathbf{w} = \mathbf{yX}^T$$

Normal Equation

$$\mathbf{w} = (\mathbf{XX}^T)^{-1} \mathbf{yX}^T$$

The corresponding solution $\mathbf{w}$ to the normal equation is called **ordinary least squares** (OLS) solution

With larger D(degree), N(samples) or higher order regression (instead of linear) make it highly challenging to derive $\mathbf{w}$ OLS

Ordinary Least Squares (OLS) (3)

- 기울기 계산 (독립 변수가 1개인 경우)

□ x : 입력 값, y : 출력 값

$$w = \frac{\sum (x - \bar{x})(y - \bar{y})}{\sum (x - \bar{x})^2}$$

- 절편 계산 (독립 변수가 1개인 경우)

$$b = \bar{y} - (\bar{x} \times w)$$

예제: 최소 제곱법을 사용한 모델 추정

- 변수 별 평균

- 공부 시간(x): $(2+4+6+8)/4 = 5$

- 성적(y): $(81+93+91+97)/4 = 90.5$

- 기울기(w)와 절편(b) 추정

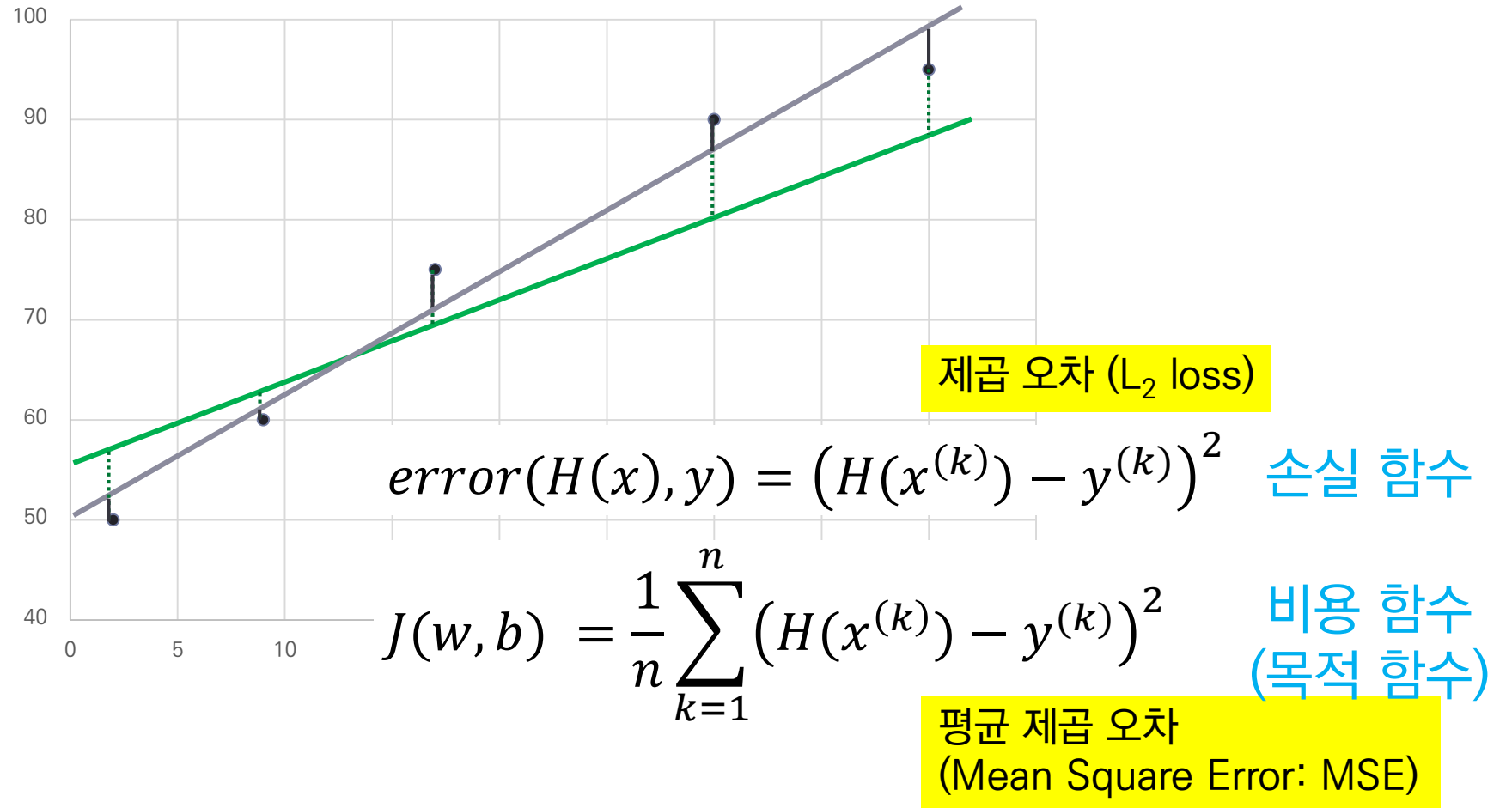
- $w = \frac{(2-5)(81-90.5) + (4-5)(93-90.5) + (6-5)(91-90.5) + (8-5)(97-90.5)}{(2-5)^2 + (4-5)^2 + (6-5)^2 + (8-5)^2} = 2.3$

- $b = 90.5 - (2.3 \times 5) = 79$

$$y = 2.3x + 79$$

9/8

손실 함수 loss function 와 비용 함수 cost function



Cost function

$$\text{error}(H(x), y) = (H(x^{(k)}) - y^{(k)})^2$$

$$H(x) = wx + b$$

$$J(w, b) = \frac{1}{n} \sum_{k=1}^n (H(x^{(k)}) - y^{(k)})^2$$

$$\hat{y}^{(k)} = H(x^{(k)})$$

$$J(w, b) = \frac{1}{n} \sum_{k=1}^n (\hat{y}^{(k)} - y^{(k)})^2$$

궁극적인 목표: 비용의 최소화

$$J(w, b) = \frac{1}{n} \sum_{k=1}^n (H(x^{(k)}) - y^{(k)})^2$$

$$\underset{w, b}{\text{minimize}} \quad J(w, b)$$

코딩으로 확인하는 평균 제곱 오차 (1)

02_Mean_Squared_Error_sykim.py

```
import numpy as np

# 예측값 계산 함수
def predict(param, x):
    return param[0]*x + param[1]

# MSE 함수
def mse(y, y_hat):
    mse_val = np.mean([(a-b)**2 for a, b in zip(y, y_hat)])
    return mse_val

# 데이터
data = [[2, 81], [4, 93], [6, 91], [8, 97]]
x = [i[0] for i in data]
y = [i[1] for i in data]
```

계속

코딩으로 확인하는 평균 제공 오차 (2)

```
# 모델 파라미터 (기울기, y 절편) 설정
param = [3, 76]

# 예측값 계산
pred = []
for n, xi in enumerate(x):
    p = predict(param, xi)
    pred.append(p)
    print("공부시간=%.f, 실제점수=%.f, 예측점수=%.f" % (xi, y[n], p))

# MSE 출력
print("MSE: %.2f" % mse(y, pred))
```

코딩으로 확인하는 평균 제곱 오차 (3)

공부시간=2, 실제점수=81, 예측점수=82
공부시간=4, 실제점수=93, 예측점수=88
공부시간=6, 실제점수=91, 예측점수=94
공부시간=8, 실제점수=97, 예측점수=100
MSE 최종값: **11.0**

param = [3, 76]

공부시간=2, 실제점수=81, 예측점수=84
공부시간=4, 실제점수=93, 예측점수=88
공부시간=6, 실제점수=91, 예측점수=93
공부시간=8, 실제점수=97, 예측점수=97
MSE 최종값: **8.299999999999985**

param = [2.3, 79]

Ridge와 Lasso

$$H(x) = wx + b$$

$$J(w, b) = \frac{1}{n} \sum_{k=1}^n \left(H(x^{(k)}) - y^{(k)} \right)^2 + \lambda \sum_{k=1}^n w_k^2$$

Ridge

규제(Regularization)

$$J(w, b) = \frac{1}{n} \sum_{k=1}^n \left(H(x^{(k)}) - y^{(k)} \right)^2 + \lambda \sum_{k=1}^n |w_k|$$

Lasso

Elastic net

참고자료

- 모두의 딥러닝, 조태호 지음, 길벗
- 케라스 창시자에게 배우는 딥러닝, 프랑소와 솔레, 길벗
- 모두를 위한 머신러닝/딥러닝 (홍콩과기대 김성훈교수)
 - <http://hunkim.github.io/ml/>
- Stanford Machine Learning
 - <http://www.holehouse.org/mlclass/>
- Machine Learning (한성대 지준교수)
 - <http://jun.hansung.ac.kr/ml>
- Regression 원리
 - <https://sanghyu.tistory.com/12>

다음 강의 주제

- 경사 하강법 (gradient descent)